Program:

```
class ParkingSystem {

    private int[] slots;

    public ParkingSystem(int big, int medium, int small) {
        slots = new int[4];
        slots[1] = big;
        slots[2] = medium;
        slots[3] = small;
    }

    public boolean addCar(int carType) {
        if (slots[carType] > 0) {
            slots[carType]--;
            return true;
        }
        return false;
    }
}
```

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

1603. Design Parking System

Solved

Easy Topics Companies Hint

Design a parking system for a parking lot. The parking lot has three kinds of parking spaces: big, medium, and small, with a fixed number of slots for each size.

Implement the `ParkingSystem` class:

- `ParkingSystem(int big, int medium, int small)` Initializes object of the `ParkingSystem` class. The number of slots for each parking space are given as part of the constructor.
- `bool addCar(int carType)` Checks whether there is a parking space of `carType` for the car that wants to get into the parking lot. `carType` can be of three kinds: big, medium, or small, which are represented by `1`, `2`, and `3` respectively. **A car can only park in a parking space of its** `carType`. If there is no space available, return `false`, else park the car in that size space and return `true`.

2.1K 70

42 Online

Code

Java Auto

```java
class ParkingSystem {

    private int[] slots;

    public ParkingSystem(int big, int medi
        slots = new int[4];
        slots[1] = big;
        slots[2] = medium;
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

["ParkingSystem","addCar","addCar","add

[[1,1,0],[1],[2],[3],[1]]

Shajil1

Access all features with our Premium subscription!

My Lists Notebook Progress

Points

Try New Features

Orders

My Playgrounds

Settings

Program:

```
import java.util.*;

class UndergroundSystem {

    class CheckIn {
        String station;
        int time;

        CheckIn(String station, int time) {
            this.station = station;
            this.time = time;
        }
    }

    class Trip {
        int totalTime;
        int count;

        Trip() {
            totalTime = 0;
            count = 0;
        }
    }

    private Map<Integer, CheckIn> checkInMap;
    private Map<String, Trip> tripMap;

    public UndergroundSystem() {
        checkInMap = new HashMap<>();
        tripMap = new HashMap<>();
    }

    public void checkIn(int id, String stationName, int t) {
        checkInMap.put(id, new CheckIn(stationName, t));
    }

    public void checkOut(int id, String stationName, int t) {
        CheckIn check = checkInMap.get(id);

        String route = check.station + "->" + stationName;
        Trip trip = tripMap.getOrDefault(route, new Trip());
```

```
        trip.totalTime += (t - check.time);
        trip.count++;

        tripMap.put(route, trip);
        checkInMap.remove(id);
    }

    public double getAverageTime(String startStation, String endStation) {
        String route = startStation + "->" + endStation;
        Trip trip = tripMap.get(route);
        return (double) trip.totalTime / trip.count;
    }
}
```

Program:

```
class MyHashSet {

    private boolean[] set;

    public MyHashSet() {
        set = new boolean[1000001];
    }

    public void add(int key) {
        set[key] = true;
    }

    public void remove(int key) {
        set[key] = false;
    }

    public boolean contains(int key) {
        return set[key];
    }
}
```

Program:

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) {
            return false;
        }

        int[] count = new int[26];

        for (int i = 0; i < s.length(); i++) {
            count[s.charAt(i) - 'a']++;
            count[t.charAt(i) - 'a']--;
        }

        for (int c : count) {
            if (c != 0) {
                return false;
            }
        }

        return true;
    }
}
```

Program:

```
import java.util.concurrent.CountDownLatch;

class Foo {

    private CountDownLatch firstDone;
    private CountDownLatch secondDone;

    public Foo() {
        firstDone = new CountDownLatch(1);
        secondDone = new CountDownLatch(1);
    }

    public void first(Runnable printFirst) throws InterruptedException {
        printFirst.run();
        firstDone.countDown();
    }

    public void second(Runnable printSecond) throws InterruptedException {
        firstDone.await();
        printSecond.run();
        secondDone.countDown();
    }

    public void third(Runnable printThird) throws InterruptedException {
        secondDone.await();
        printThird.run();
    }
}
```

Description | Editorial | Solutions | Submissions

1396. Design Underground System

Solved

Medium | Topics | Companies | Hint

An underground railway system is keeping track of customer travel times between different stations. They are using this data to calculate the average time it takes to travel from one station to another.

Implement the `UndergroundSystem` class:

- `void checkIn(int id, string stationName, int t)`
  - A customer with a card ID equal to `id`, checks in at the station `stationName` at time `t`.
  - A customer can only be checked into one place at a time.
- `void checkOut(int id, string stationName, int t)`
  - A customer with a card ID equal to `id`, checks out from the station `stationName` at time `t`.
- `double getAverageTime(string startStation, string endStation)`
  - Returns the average time it takes to travel from `startStation` to

3.6K | 75 | 20 Online

Code

Java | Auto

```java
import java.util.*;

class UndergroundSystem {

    class CheckIn {
        String station;
        int time;
```

Saved | Ln 10, Col 36

Shajil1

Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

Try New Features

Orders

My Playgrounds

Settings

Testcase | Test Result

Accepted Runtime: 23 ms

Case 1 | Case 2

Input

["UndergroundSystem","checkIn", … ckOu
t","checkOut","getAverageTime", … ageT
ime","checkOut","getAverageTime"

Program:

```
class Solution {
    public boolean checkStraightLine(int[][] coordinates) {
        int x1 = coordinates[0][0];
        int y1 = coordinates[0][1];
        int x2 = coordinates[1][0];
        int y2 = coordinates[1][1];

        for (int i = 2; i < coordinates.length; i++) {
            int x = coordinates[i][0];
            int y = coordinates[i][1];

            if ((y2 - y1) * (x - x1) != (y - y1) * (x2 - x1)) {
                return false;
            }
        }

        return true;
    }
}
```

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

1114. Print in Order

Solved

Easy | Topics | Companies

Suppose we have a class:

```
public class Foo {
  public void first() { print("first"); }
  public void second() { print("second"); }
  public void third() { print("third"); }
}
```

The same instance of `Foo` will be passed to three different threads. Thread A will call `first()`, thread B will call `second()`, and thread C will call `third()`. Design a mechanism and modify the program to ensure that `second()` is executed after `first()`, and `third()` is executed after `second()`.

Note:

We do not know how the threads will be scheduled in the operating system, even though the numbers in the input seem to imply the ordering. The

1.6K | 48 | 26 Online

Code

Java | Auto

```
import java.util.concurrent.CountDownLatch

class Foo {

    private CountDownLatch firstDone;
    private CountDownLatch secondDone;

    public Foo() {
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 2 ms

Case 1 | Case 2

Input

nums =

[1,2,3]

Shajil1

Access all features with our Premium subscription!

My Lists | Notebook | Progress

Points

Try New Features

Orders

My Playgrounds

Settings

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

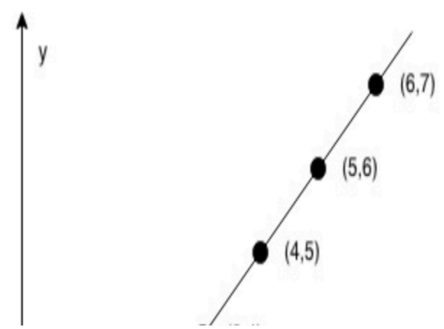
1232. Check If It Is a Straight Line

Solved

Easy | Topics | Companies | Hint

You are given an integer array `coordinates`, `coordinates[i] = [x, y]`, where `[x, y]` represents the coordinate of a point. Check if these points make a straight line in the XY plane.

Example 1:



2.7K | 79 | 12 Online

Code

Java | Auto

```java
class Solution {
    public boolean checkStraightLine(int[]
        int x1 = coordinates[0][0];
        int y1 = coordinates[0][1];
        int x2 = coordinates[1][0];
        int y2 = coordinates[1][1];

        for (int i = 2; i < coordinates.le
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

coordinates =

[[1,2],[2,3],[3,4],[4,5],[5,6],[6,7]]

Shajil1

Access all features with our Premium subscription!

My Lists | Notebook | Progress

Points

Try New Features

Orders

My Playgrounds

Settings

Proble... Submit Premium

Description | Editorial | Solutions | Submissions

705. Design HashSet

Solved

Easy Topics Companies

Design a HashSet without using any built-in hash table libraries.

Implement `MyHashSet` class:

- `void add(key)` Inserts the value `key` into the HashSet.
- `bool contains(key)` Returns whether the value `key` exists in the HashSet or not.
- `void remove(key)` Removes the value `key` in the HashSet. If `key` does not exist in the HashSet, do nothing.

Example 1:

Input
["MyHashSet", "add", "add", "contains", "contains", "add", "contains", "remove", "contains"]

4.1K 88 • 29 Online

https://leetcode.com/problemset/

Code

Java Auto

```java
class MyHashSet {

    private boolean[] set;

    public MyHashSet() {
        set = new boolean[1000001];
    }

```

Saved Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

["MyHashSet","add","add","add","remove ... ve"
,"contains","add","add","add","remove", ... ve",
"contains"]

Shajil1

Access all features with our Premium subscription!

My Lists Notebook Progress

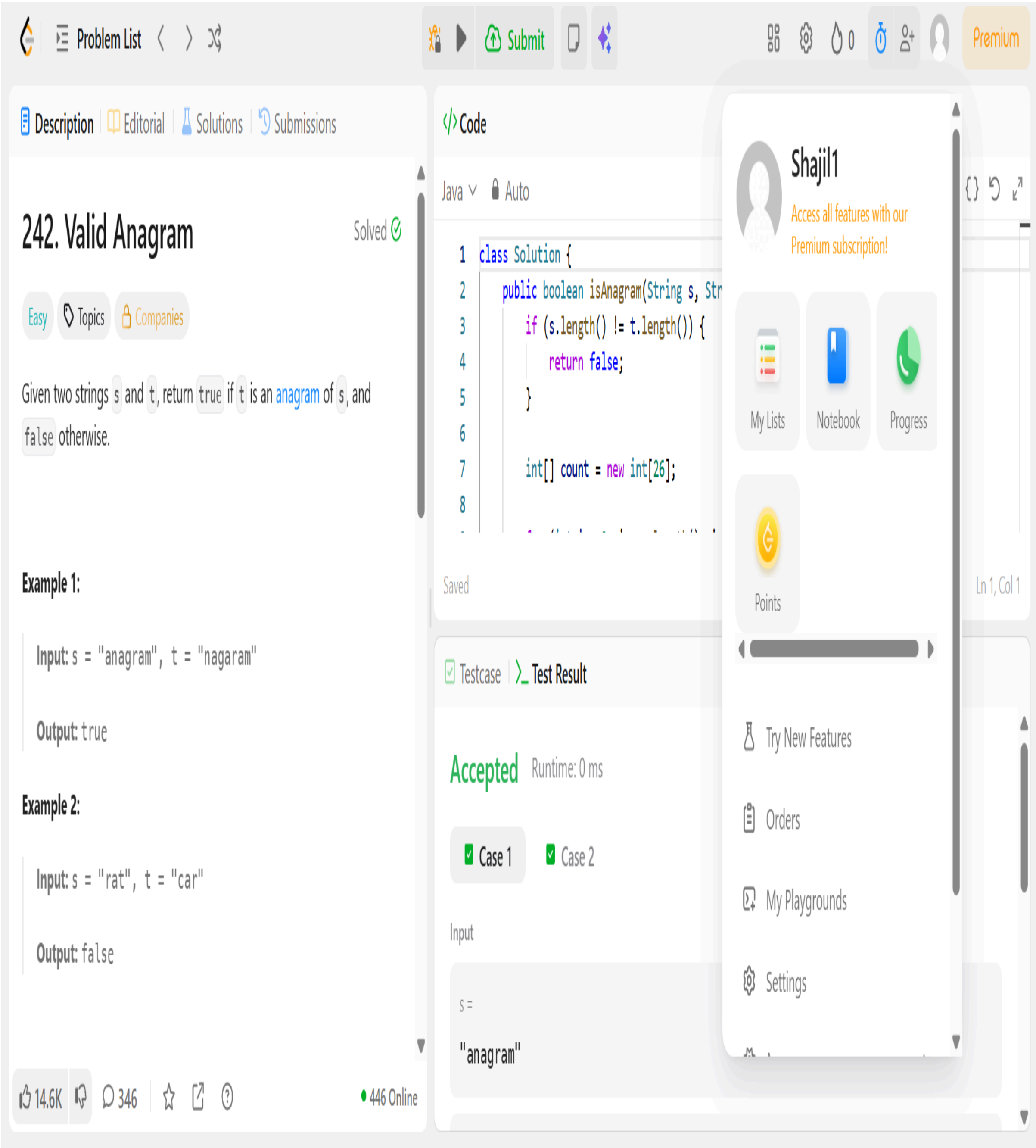
Points

Try New Features

Orders

My Playgrounds

Settings



Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
242. Valid Anagram
Solved
Easy
Topics
Companies
Given two strings s and t, return true if t is an anagram of s, and false otherwise.
Example 1:
Input: s = "anagram", t = "nagaram"
Output: true
Example 2:
Input: s = "rat", t = "car"
Output: false
14.6K
346
446 Online
Code
Java
Auto
class Solution {
public boolean isAnagram(String s, Str
if (s.length() != t.length()) {
return false;
}
int[] count = new int[26];
Saved
Ln 1, Col 1
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
s =
"anagram"
Shajil1
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings